

แบบฝึกหัด OOP ด้วย Java + Spring Boot

อ่านทีละขั้น → พิมพ์โค้ดตาม → ทำโจทย์ท้ายเอกสาร | ระดับ: เริ่มต้น ถึง กลาง

แต่ละหัวข้อมีโค้ดสั้น ๆ ให้พิมพ์ตาม พร้อมคำอธิบายว่าทำไมต้องเขียนแบบนี้

4 เสาหลักของ OOP (จำแนกก่อน)

1. Encapsulation - ซ่อนข้อมูลไว้ข้างใน เปิดเฉพาะพฤติกรรมที่จำเป็น
2. Abstraction - บอกว่าทำอะไรได้ โดยไม่ต้องบอกว่าทำอย่างไร
3. Inheritance - ดึงสิ่งที่เหมือนกันขึ้นไปไว้ที่คลาสแม่ ไม่ต้องเขียนซ้ำ
4. Polymorphism - ตัวแปรชนิดเดียว ทำงานได้หลายแบบตามวัตถุจริงที่ใส่เข้าไป

ขั้นที่ 1 Encapsulation: อย่าเปิด field ให้ใครแก้ตรง ๆ

แทนที่จะปล่อยให้โค้ดข้างนอกแก้ราคาสินค้าได้ตามใจ ให้ปิด field เป็น private แล้วเปิดเป็นเมธอดที่มีกฎอยู่ข้างใน กฎอยู่ที่เดียว แก้ทีเดียว

```
public class Product {  
    private final String name;  
    private double price;           // outsiders cannot touch this  
    public Product(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
    public void applyDiscount(double percent) {  
        if (percent < 0 || percent > 80) {           // the rule lives here  
            throw new IllegalArgumentException("invalid discount");  
        }  
        this.price = this.price * (1 - percent / 100);  
    }  
}
```

- final แปลว่าตั้งค่าได้ครั้งเดียวตอนสร้างวัตถุ เหมาะกับข้อมูลที่ไม่ควรเปลี่ยน
- ไม่มี setPrice() โดยตั้งใจ เพราะการเปลี่ยนราคาต้องผ่านกฎเสมอ

ขั้นที่ 2 Abstraction + Inheritance: ดึงส่วนที่เหมือนกันขึ้นข้างบน

Student กับ Instructor ต่างก็มีชื่อเหมือนกัน แต่บทบาทต่างกัน ให้เก็บส่วนที่เหมือนกันไว้ในคลาสแม่ แล้วให้คลาสลูกกำหนดเฉพาะส่วนที่ต่าง

```
public abstract class User {  
    private final String name;  
    protected User(String name) { this.name = name; }  
    public abstract String role();           // subclass must answer this  
    public String describe() {  
        return name + " (" + role() + ")";  
    }  
}  
public class Student extends User {  
    public Student(String name) { super(name); }  
    @Override  
    public String role() { return "STUDENT"; }  
}
```

- abstract class สร้างวัตถุตรง ๆ ไม่ได้ ต้องมีคลาสลูกมาสืบทอดเสมอ
- describe() ถูกเขียนครั้งเดียวที่คลาสแม่ แต่ผลลัพธ์เปลี่ยนตามคลาสลูก นี่คือจุดเริ่มของ polymorphism

ขั้นที่ 3 Polymorphism ผ่าน interface: สลับพฤติกรรมได้โดยไม่แก้โค้ดเดิม

interface คือสัญญาว่าทำอะไรได้ ส่วนคลาสที่ implement คือคำตอบว่าทำอย่างไร เมื่ออยากเพิ่มช่องทางส่งข้อความใหม่ ให้เขียนคลาสใหม่ ไม่ต้องไปแก้ของเดิม

```
public interface NotificationSender {
    void send(User user, String message);
}
@Component
public class EmailSender implements NotificationSender {
    @Override
    public void send(User user, String message) {
        System.out.println("EMAIL -> " + user.describe() + " : " + message);
    }
}
```

- อยากได้ LineSender หรือ SmsSender ก็ก๊อปปี้คลาสเดิม เปลี่ยนแค่ข้อความ แล้วติด @Component เหมือนกัน

ขั้นที่ 4 ประกอบร่างใน Spring Boot

Spring Boot ไม่ได้บังคับให้เขียน OOP แบบใหม่ แต่มันใช้ประโยชน์จาก OOP เต็ม ๆ คือนิยาม interface เข้ามาทำให้ไม่ใช่คลาสจริง โค้ดของเราจึงไม่ผูกกับ implementation ตัวใดตัวหนึ่ง

```
@Service
public class EnrollmentService {
    private final List<NotificationSender> senders; // Spring injects all
    public EnrollmentService(List<NotificationSender> senders) {
        this.senders = senders; // constructor injection
    }
    public void enroll(User user, String course) {
        String message = "enrolled in " + course;
        senders.forEach(sender -> sender.send(user, message));
    }
}

@RestController
public class EnrollmentController {
    private final EnrollmentService service; // inject the service
    public EnrollmentController(EnrollmentService s) { this.service = s; }
}
```

- ใช้ constructor injection ไม่ใช่ @Autowired บน field เพราะทำให้ field เป็น final และเขียนทดสอบง่ายกว่า
- รับเป็น List<NotificationSender> Spring จะรวบรวม bean ทุกตัวที่ implement interface นี้มาให้เอง อยากเพิ่ม SmsSender ก็สร้างคลาสใหม่ติด @Component จบ

กติกาง่าย ๆ ที่ใช้ตัดสินใจได้จริง

ใช้ interface เมื่อพฤติกรรมมีได้หลายแบบ เช่น ช่องทางแจ้งเตือน วิธีคิดราคา

ใช้ abstract class เมื่อมี state หรือโค้ดที่อยากแชร์ให้คลาสลูกใช้ซ้ำ

ถ้าสืบทอดแล้วต้องเขียนทับเกือบทุกเมธอด แปลว่าไม่ควรสืบทอด ให้ประกอบวัตถุแทน

แบบฝึกหัด

ข้อ 1 Encapsulation

สร้างคลาส BankAccount ที่มี balance เป็น private มีเมธอด deposit และ withdraw โดย withdraw ต้องโยน exception เมื่อเงินไม่พอ และ deposit ต้องไม่รับค่าติดลบ

ข้อ 2 Inheritance

เพิ่มคลาส Instructor extends User และเขียนเมธอด role() ให้คืนค่า INSTRUCTOR จากนั้นสร้าง List<User> ที่มีทั้ง Student และ Instructor แล้ววนพิมพ์ describe() ทีละตัว

ข้อ 3 Polymorphism + Spring

สร้าง SmsSender เพิ่มอีกหนึ่งตัว แล้วยิง endpoint POST /api/enrollments

ตรวจว่าข้อความถูกส่งครบทุกช่องทางโดยที่ไม่ได้แก้ EnrollmentService เลย ต่อยอด: ส่งเฉพาะช่องทางที่ผู้ใช้เลือก ใ้คือเพิ่ม supports(User user) ใน interface แล้วรองรับด้วย stream

เช็กลิสต์ก่อนบอกว่าเสร็จ

- ไม่มี public field เหลืออยู่ dependency ทุกตัวเป็น final และ Controller ไม่มี business logic
- เพิ่มพฤติกรรมใหม่ได้โดยไม่ต้องแก้ไฟล์เดิม ถ้ายังต้องแก้ แปลว่า abstraction ยังไม่ดีพอ